

Classification of Pets – PROJECT

Riya Harugop

[illegible]

Jupyter pet_classifier Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help Trusted

```
[10]: !pip install tensorflow
import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array

^C
WARNING: Connection timed out while downloading.
WARNING: Attempting to resume incomplete download (0 bytes/212 kB, attempt 1)
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
streamlit 1.51.0 requires protobuf<7,>=3.20, but you have protobuf 7.35.0 which is incompatible.
Collecting tensorflow
  Using cached tensorflow-2.21.0-cp313-cp313-win_amd64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Using cached absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gastl=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Using cached gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Using cached libclang-18.1.1-py2.py3-none-win_amd64.whl.metadata (5.3 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in c:\users\riyah\anaconda3\lib\site-packages (from tensorflow) (25.0)
Collecting setuptools<80.0.0, >=6.1.1 (from tensorflow)
```

Jupyter pet_classifier Last Checkpoint: 4 minutes ago

[illegible]


```
[20]: features = []
labels = []

IMAGE_SIZE = (224, 224)

for i, name in enumerate(image_names):

    label = ' '.join(name.split('_')[::-1])

    label_encoded = label_to_id[label]

    img = load_img(os.path.join(images_fp, name))

    img = tf.image.resize_with_pad(
        img_to_array(img, dtype='uint8'),
        *IMAGE_SIZE
    ).numpy().astype('uint8')

    features.append(img)
    labels.append(label_encoded)

    if i % 500 == 0:
        print(f"Processed {i}/{len(image_names)}")

print("Done")
```

```
Processed 500/7390
Processed 1000/7390
Processed 1500/7390
Processed 2000/7390
Processed 2500/7390
Processed 3000/7390
Processed 3500/7390
Processed 4000/7390
Processed 4500/7390
Processed 5000/7390
Processed 5500/7390
Processed 6000/7390
Processed 6500/7390
Processed 7000/7390
Done
```

```
[21]: print(len(features))
print(len(labels))

7390
7390
```

```
[22]: features
```

```
[22]: array([[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0],
         ...,
         [0, 0, 0],
```




```
[27]: from sklearn.model_selection import train_test_split

[32]: X_train, X_test, y_train, y_test = train_test_split(
        features_array,
        labels_one_hot.values,
        test_size=0.2,
        random_state=42
    )

[33]: X_train, X_val, y_train, y_val = train_test_split(
        X_train,
        y_train,
        test_size=0.25,
        random_state=1
    )

[34]: from tensorflow.keras import layers, Input, Model
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.applications import ResNet50
        from tensorflow.keras.applications.resnet50 import preprocess_input as pp_i
        from tensorflow.keras.layers import RandomFlip, RandomRotation, Dense, Dropout
        from tensorflow.keras.losses import CategoricalCrossentropy
        from tensorflow.keras.optimizers import Adam
```



```
[36]: data_augmentation = Sequential([
        RandomFlip("horizontal_and_vertical"),
        RandomRotation(0.2)
    ])

    prediction_layer = Dense(37, activation='softmax')

[37]: resnet_model = ResNet50(
        include_top=False,
        pooling='avg',
        weights='imagenet'
    )

    resnet_model.trainable = False

    Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
    94765736/94765736 — 24s 0us/step

[38]: # Build Model

    inputs = Input(shape=(224, 224, 3))

    x = data_augmentation(inputs)
    x = pp_i(x)
    x = resnet_model(x, training=False)
    x = Dropout(0.2)(x)
```

jupyter pet_classifier Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * Anaconda Toolbox

```
x = data_augmentation(inputs)
x = pp_i(x)
x = resnet_model(x, training=False)
x = Dropout(0.2)(x)

outputs = prediction_layer(x)

model = Model(inputs, outputs)

[39]: model.compile(optimizer=Adam(), loss=CategoricalCrossentropy(), metrics=['accuracy'])

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

[40]: history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=10
)

Epoch 1/10
139/139 ----- 768s 5s/step - accuracy: 0.4558 - loss: 1.9766 - val_accuracy: 0.8410 - val_loss: 0.5700
Epoch 2/10
139/139 ----- 745s 5s/step - accuracy: 0.7134 - loss: 0.9205 - val_accuracy: 0.8681 - val_loss: 0.4309
Epoch 3/10
139/139 ----- 664s 5s/step - accuracy: 0.7643 - loss: 0.7566 - val_accuracy: 0.8877 - val_loss: 0.3671
Epoch 4/10
```

jupyter pet_classifier Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * Anaconda Toolbox

```
validation_data=(X_val, y_val),
epochs=10
)

Epoch 1/10
139/139 ----- 768s 5s/step - accuracy: 0.4558 - loss: 1.9766 - val_accuracy: 0.8410 - val_loss: 0.5700
Epoch 2/10
139/139 ----- 745s 5s/step - accuracy: 0.7134 - loss: 0.9205 - val_accuracy: 0.8681 - val_loss: 0.4309
Epoch 3/10
139/139 ----- 664s 5s/step - accuracy: 0.7643 - loss: 0.7566 - val_accuracy: 0.8877 - val_loss: 0.3671
Epoch 4/10
139/139 ----- 683s 5s/step - accuracy: 0.7866 - loss: 0.6559 - val_accuracy: 0.8877 - val_loss: 0.3609
Epoch 5/10
139/139 ----- 1036s 7s/step - accuracy: 0.8133 - loss: 0.5932 - val_accuracy: 0.8667 - val_loss: 0.4003
Epoch 6/10
139/139 ----- 824s 6s/step - accuracy: 0.8309 - loss: 0.5228 - val_accuracy: 0.8843 - val_loss: 0.3617
Epoch 7/10
139/139 ----- 689s 5s/step - accuracy: 0.8381 - loss: 0.4979 - val_accuracy: 0.8769 - val_loss: 0.3950
Epoch 8/10
139/139 ----- 779s 5s/step - accuracy: 0.8403 - loss: 0.4963 - val_accuracy: 0.9019 - val_loss: 0.3184
Epoch 9/10
139/139 ----- 740s 5s/step - accuracy: 0.8618 - loss: 0.4326 - val_accuracy: 0.9032 - val_loss: 0.3140
Epoch 10/10
139/139 ----- 1028s 7s/step - accuracy: 0.8563 - loss: 0.4342 - val_accuracy: 0.8924 - val_loss: 0.3168

[41]: acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
```



```
[41]: acc = history.history['accuracy']
      val_acc = history.history['val_accuracy']

      loss = history.history['loss']
      val_loss = history.history['val_loss']

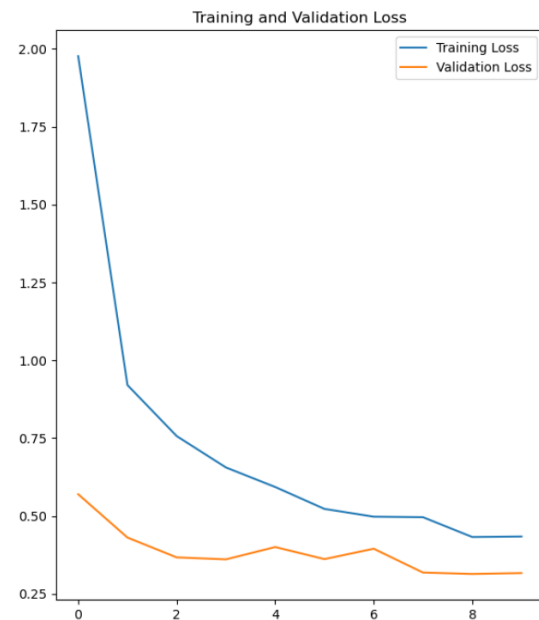
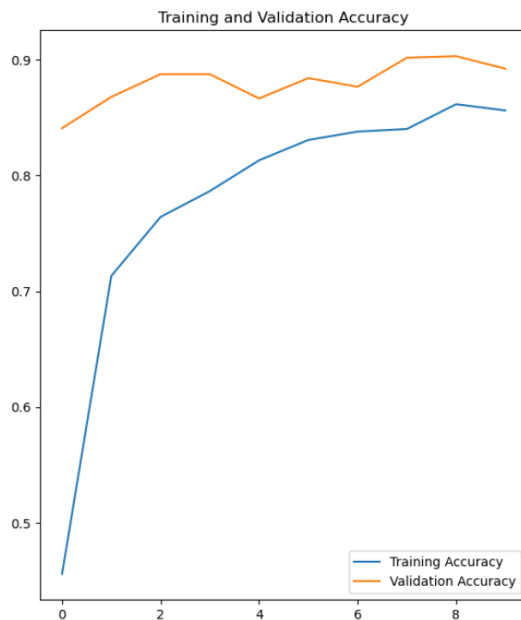
      epochs_range = range(len(acc))

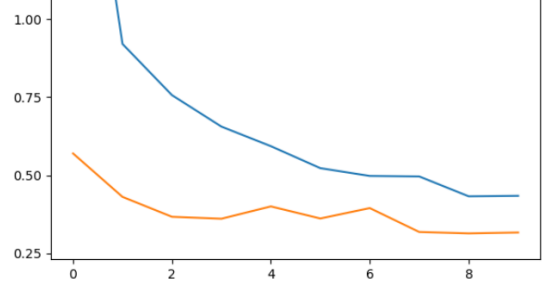
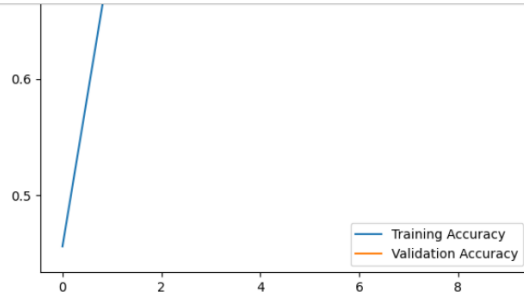
      plt.figure(figsize=(15, 8))

      plt.subplot(1, 2, 1)
      plt.plot(epochs_range, acc, label='Training Accuracy')
      plt.plot(epochs_range, val_acc, label='Validation Accuracy')
      plt.legend(loc='lower right')
      plt.title('Training and Validation Accuracy')

      plt.subplot(1, 2, 2)
      plt.plot(epochs_range, loss, label='Training Loss')
      plt.plot(epochs_range, val_loss, label='Validation Loss')
      plt.legend(loc='upper right')
      plt.title('Training and Validation Loss')

      plt.show()
```





```
[42]: model.evaluate(X_test, y_test)
47/47 ----- 279s 6s/step - accuracy: 0.8843 - loss: 0.3340
[42]: [0.3340424597263336, 0.884303092956543]
```

```
[43]: y_pred = model.predict(X_test)
47/47 ----- 309s 6s/step
```

[]:

